

Security

These notes cover common Computer Networks security topics for placements: TLS/SSL, encryption, certificates, certificate authorities, hashing, MITM attacks, firewalls, and VPNs.

1. Why Network Security Matters

Network security protects data and systems while data is being stored, processed, and transmitted across networks.

Core goals:

- **Confidentiality:** Only authorized parties can read the data.
- **Integrity:** Data is not modified without detection.
- **Authentication:** Parties prove their identity.
- **Authorization:** Authenticated users get only permitted access.
- **Availability:** Systems remain accessible when needed.
- **Non-repudiation:** A sender cannot later deny sending a message.

Common threats:

- Eavesdropping
- Packet sniffing
- Man-in-the-middle attacks
- Spoofing
- Replay attacks
- Malware
- Phishing
- Denial-of-service attacks
- Session hijacking
- DNS poisoning
- Weak passwords
- Misconfigured firewalls

Security is usually built in layers. No single mechanism is enough.

Example:

- TLS protects data in transit.
- Hashing checks integrity.
- Firewalls filter traffic.
- VPNs protect traffic over untrusted networks.
- Certificates authenticate servers.
- Access control limits who can use resources.

2. Cryptography Basics

Cryptography is the science of securing information using mathematical techniques.

It is used for:

- Encrypting data
- Authenticating users and servers
- Verifying message integrity
- Creating digital signatures
- Securing web traffic
- Protecting passwords
- Building VPN tunnels

Important terms:

- **Plaintext:** Original readable data.
- **Ciphertext:** Encrypted unreadable data.
- **Encryption:** Plaintext to ciphertext.
- **Decryption:** Ciphertext to plaintext.
- **Key:** Secret or public value used in cryptographic operations.
- **Cipher:** Algorithm used for encryption or decryption.
- **Cryptanalysis:** Study of breaking cryptographic systems.

Cryptography does not usually hide the fact that communication is happening. It hides the content and can verify identities and integrity.

3. Symmetric Encryption

Symmetric encryption uses the same key for encryption and decryption.

```
Plaintext + Secret Key -> Ciphertext  
Ciphertext + Same Secret Key -> Plaintext
```

Examples:

- AES
- ChaCha20
- DES, old and insecure
- 3DES, deprecated

AES

AES stands for Advanced Encryption Standard.

Common key sizes:

- AES-128
- AES-192
- AES-256

AES is widely used in:

- TLS
- VPNs
- Disk encryption
- Wi-Fi security
- Secure file storage

AES is fast and secure when used correctly.

Advantages of Symmetric Encryption

- Very fast
- Suitable for large data
- Efficient for real-time communication
- Uses less computation than asymmetric encryption

Disadvantages of Symmetric Encryption

- Key distribution problem: both parties need the same secret key.
- If the key is leaked, all encrypted data using that key may be compromised.
- Does not directly provide identity verification.
- Hard to manage keys among many users.

Placement Interview Point

Symmetric encryption is used for bulk data encryption because it is fast. Asymmetric encryption is often used first to safely agree on or exchange a symmetric session key.

4. Asymmetric Encryption

Asymmetric encryption uses a pair of keys:

- **Public key:** Shared openly.
- **Private key:** Kept secret.

Data encrypted with one key can be decrypted only with the other matching key, depending on the algorithm and use case.

Common algorithms:

- RSA
- Diffie-Hellman
- Elliptic Curve Diffie-Hellman
- Elliptic Curve Cryptography

Public Key Encryption

If Alice wants to send Bob a secret message:

1. Bob shares his public key.
2. Alice encrypts the message using Bob's public key.
3. Bob decrypts it using his private key.

Only Bob can decrypt it because only Bob has the private key.

Digital Signatures

Digital signatures provide authentication, integrity, and non-repudiation.

Basic idea:

1. Sender hashes the message.
2. Sender signs the hash using their private key.
3. Receiver verifies the signature using sender's public key.

If verification succeeds:

- The message came from the claimed sender.
- The message was not modified.

Digital signatures are used in:

- TLS certificates
- Software updates
- Code signing
- Emails
- Blockchain systems
- Legal digital documents

Advantages of Asymmetric Encryption

- Solves the key distribution problem better than symmetric encryption.
- Enables digital signatures.
- Supports authentication.
- Public keys can be distributed openly.

Disadvantages of Asymmetric Encryption

- Slower than symmetric encryption.
- Uses more CPU.
- Not ideal for encrypting large amounts of data directly.
- Requires trust infrastructure such as certificates and CAs.

Symmetric vs Asymmetric Encryption

Feature	Symmetric Encryption	Asymmetric Encryption
Keys used	One shared secret key	Public/private key pair
Speed	Fast	Slower
Best for	Bulk data encryption	Key exchange, signatures, authentication
Key distribution	Difficult	Easier
Examples	AES, ChaCha20	RSA, ECC, Diffie-Hellman

Feature	Symmetric Encryption	Asymmetric Encryption
Used in TLS	Encrypts actual data	Authenticates and helps establish session keys

5. Hashing

Hashing converts input data of any size into a fixed-size output called a hash, digest, or fingerprint.

Input -> Hash Function -> Fixed-size Digest

Examples:

- SHA-256
- SHA-384
- SHA-512
- SHA-3
- MD5, broken
- SHA-1, broken for collision resistance

Properties of a Good Cryptographic Hash

1. **Deterministic**

Same input always gives the same hash.

2. **Fixed output size**

Output length is fixed regardless of input size.

3. **Fast to compute**

Hash calculation should be efficient.

4. **Preimage resistance**

Given a hash, it should be infeasible to find the original input.

5. **Second preimage resistance**

Given one input, it should be infeasible to find another input with the same hash.

6. **Collision resistance**

It should be infeasible to find any two different inputs with the same hash.

7. **Avalanche effect**

Small input change causes a drastically different hash.

Hashing Is Not Encryption

Hashing is one-way. It cannot be decrypted.

Encryption is reversible if you have the correct key.

Concept	Hashing	Encryption
Reversible?	No	Yes
Uses key?	Usually no	Yes
Output	Digest	Ciphertext
Purpose	Integrity, fingerprints, password storage	Confidentiality
Example	SHA-256	AES

Uses of Hashing

- File integrity checking
- Password storage
- Digital signatures
- Message authentication codes
- Blockchain
- Certificate fingerprints
- Data deduplication
- Checksums, though not all checksums are cryptographic

Password Hashing

Passwords should not be stored in plaintext.

Bad:

```
password = "mypassword123"
```

Better:

```
hash = Hash(password + salt)
```

Best:

Use slow password hashing algorithms:

- bcrypt
- scrypt
- Argon2
- PBKDF2

Important terms:

- **Salt:** Random value added to each password before hashing.

- **Pepper**: Secret value stored separately from the database.
- **Work factor**: Controls how slow the hashing algorithm is.

Why salt matters:

- Prevents two users with the same password from having the same hash.
- Defends against precomputed rainbow table attacks.

Hash vs MAC vs Digital Signature

Mechanism	Uses Secret?	Provides Integrity?	Provides Authentication?	Non-repudiation?
Hash	No	Yes, if trusted hash is known	No	No
MAC/HMAC	Shared secret key	Yes	Yes, between parties sharing key	No
Digital signature	Private key/public key	Yes	Yes	Yes

6. Message Authentication Code and HMAC

A MAC verifies both message integrity and authenticity using a shared secret key.

HMAC means Hash-based Message Authentication Code.

```
HMAC = Hash(secret key + message, structured securely)
```

HMAC is used in:

- APIs
- TLS
- VPNs
- Signed cookies
- Request authentication

If an attacker changes the message, they cannot generate a valid HMAC without the secret key.

7. TLS and SSL

TLS stands for Transport Layer Security.

SSL stands for Secure Sockets Layer.

SSL is the older protocol. It is considered insecure and deprecated. Modern secure communication uses TLS, but people still commonly say "SSL" informally.

Common versions:

- SSL 2.0: insecure
- SSL 3.0: insecure
- TLS 1.0: deprecated
- TLS 1.1: deprecated
- TLS 1.2: widely used
- TLS 1.3: modern, faster, more secure

TLS is used to secure:

- HTTPS
- Email protocols such as SMTPS, IMAPS, POP3S
- APIs
- VPNs
- Messaging apps
- Database connections

What TLS Provides

TLS provides:

- Confidentiality through encryption
- Integrity through MAC or AEAD ciphers
- Authentication through certificates
- Protection against many MITM attacks
- Secure session key establishment

TLS usually runs above TCP and below application protocols.

```
Application Layer: HTTP
Security Layer: TLS
Transport Layer: TCP
Network Layer: IP
```

HTTPS is HTTP over TLS.

```
HTTP + TLS = HTTPS
```

TLS Handshake

The TLS handshake is the process where client and server agree on security parameters.

High-level TLS handshake:

1. Client says hello and sends supported TLS versions and cipher suites.
2. Server chooses TLS version and cipher suite.
3. Server sends its certificate.
4. Client verifies the certificate.

5. Client and server perform key exchange.
6. Both derive shared symmetric session keys.
7. Encrypted communication begins.

After the handshake, symmetric encryption is used for actual data transfer because it is fast.

TLS 1.2 Simplified Flow

1. **ClientHello**

Client sends:

- Supported TLS versions
- Supported cipher suites
- Random value
- Extensions such as SNI

2. **ServerHello**

Server replies with:

- Chosen TLS version
- Chosen cipher suite
- Random value

3. **Certificate**

Server sends its certificate chain.

4. **Key Exchange**

Client and server establish a shared secret.

5. **Finished Messages**

Both confirm that the handshake was successful.

6. **Encrypted Data**

Application data is encrypted using session keys.

TLS 1.3 Improvements

TLS 1.3:

- Removes old insecure algorithms.
- Reduces handshake latency.
- Supports faster connection setup.
- Encrypts more handshake data.
- Uses forward secrecy by default.
- Removes RSA key exchange.
- Supports 0-RTT data, with replay risk.

Cipher Suite

A cipher suite is a set of algorithms used by TLS.

In TLS 1.2, a cipher suite may specify:

- Key exchange algorithm
- Authentication algorithm
- Bulk encryption algorithm
- MAC or AEAD algorithm

Example:

```
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
```

Meaning:

- ECDHE: Key exchange
- RSA: Authentication
- AES_128_GCM: Symmetric encryption with authenticated encryption
- SHA256: Hash function used in handshake

TLS 1.3 cipher suites are simpler because key exchange and authentication are handled separately.

Forward Secrecy

Forward secrecy means that even if a server's long-term private key is compromised later, past session traffic remains safe.

It is commonly achieved using ephemeral Diffie-Hellman:

- DHE
- ECDHE

Why it matters:

If an attacker records encrypted traffic today and steals the server private key later, they still cannot decrypt old traffic if forward secrecy was used.

SNI

SNI stands for Server Name Indication.

It lets the client tell the server which hostname it wants during the TLS handshake.

Why it is needed:

One server IP address may host many domains. SNI allows the server to choose the correct certificate.

Example:

- `example.com`
- `api.example.com`
- `shop.example.com`

All may be on the same IP, but each may need a different certificate.

ALPN

ALPN stands for Application-Layer Protocol Negotiation.

It lets client and server decide which application protocol to use over TLS.

Examples:

- HTTP/1.1
- HTTP/2
- HTTP/3, though HTTP/3 uses QUIC over UDP with TLS 1.3 integration

Common TLS Attacks and Issues

- Expired certificates
- Self-signed certificates not trusted by clients
- Weak cipher suites
- Old TLS versions enabled
- Certificate hostname mismatch
- Missing intermediate certificates
- Private key leakage
- Downgrade attacks
- Misconfigured redirects from HTTP to HTTPS
- Mixed content on websites

HTTPS Does Not Hide Everything

HTTPS encrypts:

- URL path and query after TLS starts
- Request body
- Response body
- Cookies
- Headers

HTTPS may not fully hide:

- Server IP address
- Domain name through DNS unless encrypted DNS is used
- SNI unless Encrypted Client Hello is used
- Traffic timing
- Amount of data transferred

8. Certificates

A digital certificate binds a public key to an identity.

In TLS, a certificate helps prove that a server really owns a domain.

A certificate usually contains:

- Subject name
- Public key
- Issuer
- Serial number
- Validity period
- Signature algorithm
- Digital signature by issuer
- Subject Alternative Names
- Key usage extensions

Important fields:

- **Common Name:** Older identity field.
- **Subject Alternative Name:** Modern field used for domain validation.
- **Issuer:** CA that issued the certificate.
- **Validity period:** Not before and not after dates.
- **Public key:** Server's public key.
- **Signature:** CA's signature over certificate data.

Certificate Chain

Certificates are usually verified through a chain of trust.

```
Root CA
  -> Intermediate CA
      -> Server Certificate
```

The browser or operating system already trusts root CA certificates in its trust store.

The server sends:

- Server certificate
- Intermediate certificate or certificates

The root certificate is usually already present on the client.

Certificate Validation

When a browser connects to <https://example.com>, it checks:

1. Is the certificate signed by a trusted CA?
2. Is the certificate valid for the current date?
3. Does the certificate match the requested hostname?
4. Is the certificate revoked?

5. Is the certificate chain complete?
6. Is the certificate allowed for server authentication?
7. Are the algorithms and key sizes secure enough?

If validation fails, the browser shows a security warning.

Types of Certificates

By validation level:

- **DV certificate:** Domain Validation. Proves control over domain.
- **OV certificate:** Organization Validation. Includes organization details.
- **EV certificate:** Extended Validation. Stronger business verification, less visually emphasized by modern browsers.

By coverage:

- **Single-domain certificate:** Secures one domain.
- **Wildcard certificate:** Secures subdomains, for example `*.example.com`.
- **Multi-domain certificate/SAN certificate:** Secures multiple domain names.

Self-Signed Certificate

A self-signed certificate is signed by its own private key instead of a trusted CA.

It can encrypt traffic, but browsers do not trust it by default.

Used for:

- Local development
- Internal testing
- Private networks

Problem:

It does not prove identity to public clients unless the client explicitly trusts it.

Certificate Revocation

Revocation means invalidating a certificate before its expiry date.

Reasons:

- Private key compromised
- Certificate issued incorrectly
- Domain ownership changed
- CA discovered fraud

Revocation methods:

- **CRL:** Certificate Revocation List
- **OCSP:** Online Certificate Status Protocol
- **OCSP stapling:** Server includes fresh revocation proof during TLS handshake

Certificate Pinning

Certificate pinning means an application remembers or hardcodes expected certificate/public key information.

It can defend against rogue CA-issued certificates, but it is risky:

- If pins are wrong or not updated, the app can break.
- Recovery can be difficult.

Modern systems often prefer Certificate Transparency and careful CA management over aggressive pinning.

9. Certificate Authorities

A Certificate Authority, or CA, is a trusted organization that issues digital certificates.

Examples:

- DigiCert
- GlobalSign
- Sectigo
- Let's Encrypt
- Google Trust Services

CA responsibilities:

- Verify identity or domain control.
- Issue certificates.
- Sign certificates.
- Revoke compromised certificates.
- Maintain secure CA infrastructure.

How a CA Issues a Certificate

1. Website owner generates a key pair.
2. Website owner creates a CSR.
3. CSR is sent to CA.
4. CA verifies domain or organization.
5. CA signs and issues certificate.
6. Website installs certificate and private key on server.

CSR stands for Certificate Signing Request.

A CSR contains:

- Public key
- Domain name
- Organization information, if needed
- Signature generated using private key

The private key should never be sent to the CA.

Trust Store

A trust store is a collection of trusted root certificates.

Trust stores exist in:

- Browsers
- Operating systems
- Java runtime
- Mobile devices
- Some applications

If a root CA is removed from trust stores, certificates chaining to it may stop being trusted.

Certificate Transparency

Certificate Transparency, or CT, is a public logging system for certificates.

Purpose:

- Detect mistakenly or maliciously issued certificates.
- Allow domain owners to monitor certificate issuance.
- Improve CA accountability.

10. Diffie-Hellman Key Exchange

Diffie-Hellman lets two parties agree on a shared secret over an insecure network without directly sending the secret.

Important idea:

Even if an attacker sees the exchanged public values, they cannot easily compute the shared secret.

Variants:

- DH
- DHE, ephemeral DH
- ECDH, elliptic curve DH
- ECDHE, ephemeral elliptic curve DH

ECDHE is widely used in TLS because it is efficient and supports forward secrecy.

Diffie-Hellman alone does not authenticate the parties. Without authentication, it is vulnerable to MITM attacks.

That is why TLS combines key exchange with certificate-based authentication.

11. Man-in-the-Middle Attack

A man-in-the-middle, or MITM, attack happens when an attacker secretly intercepts and possibly modifies communication between two parties.

```
Client <-> Attacker <-> Server
```

The client thinks it is talking to the server.

The server thinks it is talking to the client.

MITM Goals

- Read sensitive data
- Modify messages
- Steal login credentials
- Inject malware
- Hijack sessions
- Downgrade encryption
- Redirect users to fake websites

Common MITM Techniques

- ARP spoofing on local networks
- DNS spoofing or poisoning
- Rogue Wi-Fi access points
- SSL stripping
- Evil twin hotspots
- BGP hijacking
- Malicious proxy servers
- Compromised routers
- Fake certificates

ARP Spoofing MITM

ARP maps IP addresses to MAC addresses in a local network.

In ARP spoofing:

1. Attacker sends fake ARP messages.
2. Victim associates gateway IP with attacker's MAC address.
3. Traffic flows through attacker.
4. Attacker forwards traffic to gateway to avoid detection.

Defense:

- Dynamic ARP inspection
- Static ARP entries for critical systems
- Network segmentation
- VPN
- HTTPS/TLS
- Switch security features

DNS Spoofing MITM

In DNS spoofing:

1. User asks for IP of a domain.
2. Attacker returns fake IP.
3. User connects to attacker's server.

Defense:

- DNSSEC
- HTTPS certificate validation
- Encrypted DNS, such as DoH or DoT
- Secure resolver configuration

SSL Stripping

SSL stripping downgrades HTTPS connections to HTTP when possible.

Example:

1. User types `example.com`.
2. Browser first tries HTTP.
3. Attacker blocks redirect to HTTPS.
4. User unknowingly uses HTTP.

Defense:

- HSTS
- HTTPS-only mode
- Never entering passwords on HTTP pages
- Secure cookies

HSTS

HSTS stands for HTTP Strict Transport Security.

It tells browsers to always use HTTPS for a domain.

Header:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

Benefits:

- Prevents SSL stripping.
- Forces HTTPS.
- Protects users from accidental HTTP access.

Detecting MITM

Signs:

- Browser certificate warning
- Certificate issuer unexpectedly changed
- HTTP instead of HTTPS
- Login pages on suspicious domains
- Network slowdown
- Duplicate IP conflicts
- Unexpected proxy settings
- Captive portal behavior

Strong defense:

- Validate certificates.
- Use HTTPS.
- Avoid unknown Wi-Fi.
- Use VPN on untrusted networks.
- Use HSTS.
- Keep systems updated.
- Use DNSSEC-aware resolvers where possible.

12. Firewalls

A firewall is a network security device or software that monitors and controls incoming and outgoing traffic based on rules.

Firewalls enforce traffic policy.

They can be:

- Hardware devices
- Software running on hosts
- Cloud security groups
- Virtual appliances

Basic Firewall Actions

- Allow
- Deny
- Drop silently
- Reject with response
- Log

Drop vs reject:

- **Drop:** Packet is silently discarded.
- **Reject:** Packet is refused and sender receives an error.

Types of Firewalls

1. Packet Filtering Firewall

Works at network and transport layers.

Filters based on:

- Source IP
- Destination IP
- Source port
- Destination port
- Protocol
- Interface

Advantages:

- Fast
- Simple
- Low overhead

Disadvantages:

- Does not understand application data
- Limited context
- Can be bypassed with allowed ports

2. Stateful Firewall

Tracks connection state.

It knows whether a packet belongs to an existing valid connection.

Example:

If an internal client opens a TCP connection to a web server, the firewall allows returning packets for that connection.

Advantages:

- More secure than stateless packet filtering
- Understands connection context

Disadvantages:

- More resource usage
- State table exhaustion can be a risk

3. Application Layer Firewall

Inspects application-level traffic.

Examples:

- Web Application Firewall
- Email security gateway

- Proxy firewall

Can detect:

- SQL injection
- Cross-site scripting
- Malicious HTTP requests
- Suspicious payloads

4. Proxy Firewall

Acts as an intermediary between client and server.

```
Client -> Proxy Firewall -> Server
```

Advantages:

- Hides internal network details
- Can inspect content
- Can enforce authentication
- Can cache content

Disadvantages:

- Adds latency
- More complex

5. Next-Generation Firewall

NGFW includes traditional firewall features plus:

- Deep packet inspection
- Application awareness
- Intrusion prevention
- User identity integration
- Malware detection
- TLS inspection

Firewall Rules

Rules are usually processed top to bottom.

Example:

Action	Source	Destination	Protocol	Port
Allow	10.0.0.0/24	Any	TCP	443
Allow	Admin IP	Server	TCP	22

Action	Source	Destination	Protocol	Port
Deny	Any	Any	Any	Any

Common best practice:

- Deny by default.
- Allow only necessary traffic.
- Log important denied traffic.
- Avoid broad rules like "allow any any".
- Review rules regularly.

Inbound vs Outbound Filtering

Inbound filtering controls traffic entering a network or host.

Outbound filtering controls traffic leaving a network or host.

Outbound filtering is important because:

- Malware may try to contact command-and-control servers.
- Data exfiltration can be blocked.
- Internal systems should not connect everywhere unnecessarily.

Network ACL vs Security Group

In cloud networking:

- **Network ACL:** Often stateless, subnet-level filtering.
- **Security group:** Often stateful, instance-level filtering.

Exact behavior depends on cloud provider.

Firewall Limitations

Firewalls cannot fully protect against:

- Attacks using allowed traffic
- Insider threats
- Social engineering
- Weak passwords
- Malware already inside the network
- Encrypted malicious traffic without inspection
- Misconfigured allowed rules

Firewalls are necessary but not sufficient.

13. VPN

VPN stands for Virtual Private Network.

A VPN creates an encrypted tunnel over an untrusted network.

User Device -> Encrypted Tunnel -> VPN Server -> Internet/Internal Network

VPNs are used for:

- Secure remote access
- Site-to-site connectivity
- Protecting traffic on public Wi-Fi
- Accessing private corporate resources
- Hiding traffic content from local network attackers
- Connecting branch offices

What a VPN Provides

- Encryption
- Authentication
- Integrity
- Secure tunneling
- Remote private network access

What a VPN Does Not Guarantee

A VPN does not automatically:

- Make you anonymous from everyone
- Protect against phishing
- Stop malware
- Secure a compromised device
- Make HTTP websites safe
- Prevent tracking by logged-in services

Your VPN provider can still see metadata and sometimes destination traffic patterns. HTTPS is still important even when using a VPN.

Remote Access VPN

Used by individual users to connect to a private network.

Example:

An employee working from home connects to the company VPN and accesses internal servers.

Site-to-Site VPN

Connects entire networks.

Example:

Branch office network connects securely to headquarters network.

Branch LAN -> VPN Gateway -> Internet -> VPN Gateway -> HQ LAN

Full Tunnel vs Split Tunnel

Full tunnel:

- All traffic goes through VPN.
- More secure control.
- More bandwidth usage.
- Can be slower.

Split tunnel:

- Only selected traffic goes through VPN.
- Internet traffic may go directly.
- Faster and less load on VPN.
- Higher risk if local network is untrusted.

Common VPN Protocols

IPsec

IPsec works at the network layer.

It can provide:

- Confidentiality
- Integrity
- Authentication
- Anti-replay protection

Important IPsec components:

- AH: Authentication Header
- ESP: Encapsulating Security Payload
- IKE: Internet Key Exchange
- Security Association

ESP is commonly used because it provides encryption and integrity.

IPsec modes:

- **Transport mode:** Encrypts payload, keeps original IP header.
- **Tunnel mode:** Encapsulates entire original IP packet inside a new IP packet.

Tunnel mode is common for VPNs.

SSL/TLS VPN

Uses TLS to create secure remote access.

Often used through:

- Browser portals
- VPN clients

Advantages:

- Works well through NAT and firewalls
- Uses TCP/443 commonly
- Convenient for remote users

OpenVPN

Open-source VPN protocol using TLS.

Can run over:

- UDP
- TCP

UDP is usually preferred for performance.

WireGuard

Modern VPN protocol.

Features:

- Simple design
- High performance
- Uses modern cryptography
- Small codebase
- Easy configuration compared to many older VPNs

L2TP/IPsec

L2TP provides tunneling, IPsec provides security.

Older but still seen in some environments.

PPTP

Old VPN protocol.

Considered insecure and should be avoided.

VPN vs HTTPS

Feature	VPN	HTTPS
Protects	Device/network traffic through tunnel	Browser/app connection to one server

Feature	VPN	HTTPS
Scope	Many applications or entire network	Specific application connection
Endpoint	VPN server	Website/server
Local Wi-Fi protection	Yes	Yes, for HTTPS traffic
Protects after VPN server?	Not necessarily	Yes, to HTTPS server
Requires trust in	VPN provider/admin	Website certificate chain

Best practice:

Use HTTPS even when connected to a VPN.

14. Attacks Related to Cryptography and Network Security

Replay Attack

An attacker captures valid data and sends it again later.

Example:

Capturing an authentication token and replaying it.

Defense:

- Nonces
- Timestamps
- Session tokens
- Sequence numbers
- TLS
- Anti-replay windows in VPNs

Downgrade Attack

An attacker forces parties to use weaker security.

Example:

Forcing TLS 1.2 or older instead of TLS 1.3, or forcing weak ciphers.

Defense:

- Disable old protocols.
- Use HSTS.
- Use secure client/server configurations.
- TLS downgrade protection.

Brute Force Attack

Trying all possible keys or passwords.

Defense:

- Strong keys
- Long passwords
- Rate limiting
- Account lockout with care
- MFA
- Slow password hashing

Dictionary Attack

Trying likely passwords from a wordlist.

Defense:

- Strong passwords
- Password managers
- Salted slow hashes
- MFA

Rainbow Table Attack

Uses precomputed hash tables to reverse password hashes.

Defense:

- Unique salts
- Slow password hashing

Birthday Attack

Uses probability to find hash collisions faster than brute force might suggest.

This is one reason hash output size matters.

Session Hijacking

Attacker steals or guesses a session identifier.

Defense:

- HTTPS everywhere
- Secure cookies
- HttpOnly cookies
- SameSite cookies
- Session expiration
- Regenerate session ID after login

DNS Poisoning

Attacker corrupts DNS responses or cache entries.

Defense:

- DNSSEC
- Source port randomization
- HTTPS certificate validation
- Trusted resolvers

15. Secure Cookies and Web Network Security

Cookies often store session identifiers.

Important cookie flags:

- **Secure:** Sent only over HTTPS.
- **HttpOnly:** Not accessible to JavaScript.
- **SameSite:** Controls cross-site sending.

Example:

```
Set-Cookie: session=abc123; Secure; HttpOnly; SameSite=Lax
```

Why this matters:

- Secure helps prevent leakage over HTTP.
- HttpOnly helps reduce impact of XSS.
- SameSite helps reduce CSRF.

16. DNS Security

DNS translates domain names to IP addresses.

Security issues:

- Spoofed DNS responses
- Cache poisoning
- Domain hijacking
- DNS tunneling
- DNS amplification attacks
- Privacy leakage

DNSSEC

DNSSEC adds digital signatures to DNS records.

It provides:

- Data origin authentication
- Integrity

It does not provide:

- Encryption
- Confidentiality

DoH and DoT

DoH: DNS over HTTPS.

DoT: DNS over TLS.

They encrypt DNS queries between client and resolver.

They improve privacy against local observers, but the resolver still sees queries.

17. Wi-Fi Security

Common Wi-Fi security standards:

- WEP: insecure
- WPA: old
- WPA2: widely used
- WPA3: modern

WEP should never be used.

WPA2 with a strong password is acceptable in many cases.

WPA3 improves protection, especially against offline password guessing.

Common Wi-Fi attacks:

- Evil twin access point
- Deauthentication attack
- Weak password cracking
- Rogue access point
- Captive portal phishing

Defenses:

- WPA2/WPA3
- Strong passwords
- Avoid unknown Wi-Fi
- Use VPN on public Wi-Fi
- Disable auto-join
- Validate HTTPS certificates

18. Network Address Translation and Security

NAT translates private IP addresses to public IP addresses.

NAT is not a firewall by itself, but it can hide internal hosts from direct inbound access.

Do not rely on NAT alone for security.

Use firewall rules, segmentation, and host security.

19. IDS and IPS

IDS stands for Intrusion Detection System.

IPS stands for Intrusion Prevention System.

IDS:

- Monitors traffic
- Detects suspicious activity
- Generates alerts
- Does not necessarily block traffic

IPS:

- Monitors traffic
- Detects suspicious activity
- Blocks or prevents attacks

Detection methods:

- Signature-based detection
- Anomaly-based detection
- Behavior-based detection

Limitations:

- False positives
- False negatives
- Encrypted traffic visibility issues
- Requires tuning

20. Zero Trust Networking

Zero Trust means do not automatically trust any user, device, or network location.

Principles:

- Verify explicitly.
- Use least privilege.
- Assume breach.
- Continuously monitor.
- Segment access.
- Enforce strong identity.

Zero Trust is not one product. It is an architecture and security mindset.

21. Common Ports for Security Topics

Protocol	Port	Notes
HTTP	80	Unencrypted web
HTTPS	443	HTTP over TLS
SSH	22	Secure remote login
Telnet	23	Insecure remote login
FTP	20/21	Insecure file transfer
SFTP	22	File transfer over SSH
FTPS	990 or explicit over 21	FTP over TLS
SMTP	25	Mail transfer
SMTPS	465	SMTP over TLS
SMTP submission	587	Often STARTTLS
DNS	53	UDP/TCP
DoT	853	DNS over TLS
DHCP	67/68	IP assignment
RDP	3389	Remote desktop
IPsec IKE	500	VPN key exchange
IPsec NAT-T	4500	IPsec through NAT
OpenVPN	1194	Common default
WireGuard	51820	Common default

22. Important Security Headers

Common HTTP security headers:

- **Strict-Transport-Security**: Forces HTTPS.
- **Content-Security-Policy**: Controls allowed content sources.
- **X-Content-Type-Options**: Prevents MIME sniffing.
- **X-Frame-Options**: Helps prevent clickjacking.
- **Referrer-Policy**: Controls referrer information.
- **Permissions-Policy**: Controls browser features.

These are web security topics, but they often appear with HTTPS and TLS questions.

23. Practical TLS Certificate Troubleshooting

Common browser errors:

- Certificate expired
- Certificate not yet valid

- Hostname mismatch
- Self-signed certificate
- Unknown issuer
- Incomplete certificate chain
- Weak signature algorithm
- Revoked certificate

Checklist:

1. Check system date and time.
2. Check certificate expiry.
3. Check Subject Alternative Name.
4. Check certificate chain.
5. Check whether intermediate certificates are installed.
6. Check if private key matches certificate.
7. Check server supports modern TLS versions.
8. Check firewall or proxy is not intercepting TLS.

24. Interview Questions and Answers

What is the difference between HTTP and HTTPS?

HTTP sends data in plaintext. HTTPS is HTTP over TLS, so it provides encryption, integrity, and server authentication.

Why is symmetric encryption faster than asymmetric encryption?

Symmetric encryption uses simpler mathematical operations and one shared key. Asymmetric encryption uses complex public-key mathematics, so it is slower and mostly used for key exchange and signatures.

Why does TLS use both symmetric and asymmetric cryptography?

TLS uses asymmetric cryptography to authenticate the server and establish shared secrets. It then uses symmetric encryption for actual data transfer because symmetric encryption is much faster.

What is a certificate?

A certificate is a digitally signed document that binds an identity, such as a domain name, to a public key.

What is a certificate authority?

A CA is a trusted entity that verifies identities or domain control and signs certificates.

What happens if a certificate expires?

Browsers and clients should reject it or show a warning because the certificate is no longer valid.

What is the difference between hashing and encryption?

Hashing is one-way and used for integrity and fingerprints. Encryption is reversible with a key and used for confidentiality.

What is a man-in-the-middle attack?

A MITM attack occurs when an attacker secretly intercepts or modifies communication between two parties.

How does HTTPS prevent MITM?

HTTPS uses TLS certificates to authenticate the server and encryption to protect data. If an attacker presents a fake certificate, the browser should reject it unless the attacker controls a trusted certificate or the user ignores warnings.

What is a firewall?

A firewall monitors and filters network traffic based on security rules.

What is a VPN?

A VPN creates an encrypted tunnel over an untrusted network, allowing secure remote access or protected communication.

What is forward secrecy?

Forward secrecy ensures past sessions remain secure even if the server's long-term private key is compromised later.

What is the role of hashing in digital signatures?

Instead of signing the full message, the sender signs the hash of the message. This is efficient and still protects integrity.

What is HSTS?

HSTS is an HTTP security policy that tells browsers to always use HTTPS for a domain.

What is the difference between IDS and IPS?

IDS detects and alerts. IPS detects and blocks.

What is the difference between authentication and authorization?

Authentication verifies identity. Authorization decides what an authenticated user is allowed to access.

What is a replay attack?

A replay attack happens when an attacker captures valid data and resends it later to trick a system.

Why is MD5 not secure?

MD5 is vulnerable to collision attacks, meaning attackers can find different inputs with the same hash.

Why should passwords be salted before hashing?

Salts prevent identical passwords from producing identical hashes and defend against rainbow table attacks.

What is the difference between a root CA and an intermediate CA?

A root CA is directly trusted by clients. An intermediate CA is signed by a root CA and signs end-entity certificates. Intermediates reduce risk because root keys can be kept more protected.

25. Quick Revision Tables

Security Mechanisms

Mechanism	Main Purpose
Encryption	Confidentiality
Hashing	Integrity/fingerprint
HMAC	Integrity and shared-key authentication
Digital signature	Integrity, authentication, non-repudiation
Certificate	Bind identity to public key
CA	Establish certificate trust
TLS	Secure communication channel
Firewall	Traffic filtering
VPN	Encrypted tunnel
IDS	Detect intrusions
IPS	Block intrusions

Algorithm Examples

Category	Examples
Symmetric encryption	AES, ChaCha20
Asymmetric encryption	RSA, ECC
Key exchange	DH, ECDH, ECDHE
Hashing	SHA-256, SHA-3
Password hashing	bcrypt, scrypt, Argon2, PBKDF2
MAC	HMAC
Digital signatures	RSA signatures, ECDSA, EdDSA

Old or Weak Technologies to Avoid

Technology	Issue
SSL	Deprecated and insecure

Technology	Issue
TLS 1.0/1.1	Deprecated
MD5	Collision attacks
SHA-1	Weak collision resistance
DES	Small key size
3DES	Deprecated, slow, legacy
WEP	Broken Wi-Fi security
PPTP	Insecure VPN
Telnet	Plaintext login
FTP	Plaintext credentials/data

26. Memory Hooks

- **Hashing is not encryption:** Hashing is one-way.
- **HTTPS = HTTP + TLS.**
- **TLS uses asymmetric first, symmetric later.**
- **Certificates prove public key ownership.**
- **CAs create trust chains.**
- **MITM is defeated by authentication plus encryption.**
- **Firewalls filter traffic but do not fix bad applications.**
- **VPN protects the path to the VPN server, not magically everything forever.**
- **Forward secrecy protects old sessions from future key theft.**
- **Salt protects password hashes from precomputed attacks.**

27. Placement-Focused One-Liners

- Confidentiality means only intended parties can read the data.
- Integrity means unauthorized modification can be detected.
- Authentication proves identity.
- Authorization controls access after identity is known.
- TLS secures application traffic over a network.
- SSL is obsolete; TLS is the modern protocol.
- A certificate binds a domain or identity to a public key.
- A CA signs certificates and is trusted through root stores.
- Symmetric encryption is fast but has key-sharing problems.
- Asymmetric encryption solves key distribution and enables signatures but is slower.
- Hash functions are one-way and fixed-size.
- HMAC uses a secret key to verify integrity and authenticity.
- Digital signatures use private keys for signing and public keys for verification.
- MITM attacks intercept communication between parties.
- HSTS helps prevent HTTPS downgrade attacks.
- Firewalls enforce traffic rules.

- Stateful firewalls track connection state.
- VPNs create encrypted tunnels over untrusted networks.
- IPsec works at the network layer.
- TLS usually works above TCP.
- DNSSEC signs DNS data but does not encrypt DNS queries.

28. Typical Scenario Explanations

Opening an HTTPS Website

1. Browser resolves domain using DNS.
2. Browser connects to server IP on port 443.
3. Browser sends TLS ClientHello.
4. Server sends certificate and handshake parameters.
5. Browser validates certificate chain and hostname.
6. Browser and server derive shared session keys.
7. HTTP requests and responses are encrypted inside TLS.

Logging Into a Website Securely

Good secure login flow:

1. User connects over HTTPS.
2. Server certificate is validated.
3. User submits password inside encrypted TLS connection.
4. Server compares password using salted slow hash.
5. Server creates secure session.
6. Session cookie uses Secure, HttpOnly, and SameSite flags.

Public Wi-Fi Risk

On public Wi-Fi, attackers may:

- Sniff traffic
- Run evil twin hotspots
- Try ARP spoofing
- Redirect DNS
- Attempt SSL stripping

Defenses:

- Use HTTPS
- Use VPN
- Avoid sensitive actions on suspicious networks
- Do not ignore certificate warnings
- Disable automatic Wi-Fi joining

29. Common Mistakes in Answers

Avoid saying:

- "SSL is the same as TLS." Better: SSL is old; TLS is modern, but SSL is often used informally.
- "Hashing encrypts passwords." Better: Passwords are hashed, not encrypted.
- "VPN makes you completely anonymous." Better: VPN shifts trust to the VPN provider and protects traffic over part of the path.
- "Firewall stops all attacks." Better: Firewall filters traffic but cannot stop every application-level or insider attack.
- "Certificate encrypts data." Better: Certificate authenticates identity and provides a public key; encryption is performed by TLS algorithms.
- "Public key decrypts everything." Better: Public/private key use depends on encryption or signature context.

30. Final Revision Checklist

Before an interview, be able to explain:

- Difference between HTTP, HTTPS, SSL, and TLS
- TLS handshake at a high level
- Why TLS uses both symmetric and asymmetric cryptography
- What certificates and CAs do
- How certificate validation works
- Difference between hashing, encryption, HMAC, and digital signatures
- Why salts are used for passwords
- What MITM attacks are and how HTTPS helps
- Difference between packet filtering, stateful firewall, and application firewall
- What VPNs do and do not protect
- Difference between IPsec VPN and TLS VPN
- Common weak algorithms and protocols to avoid